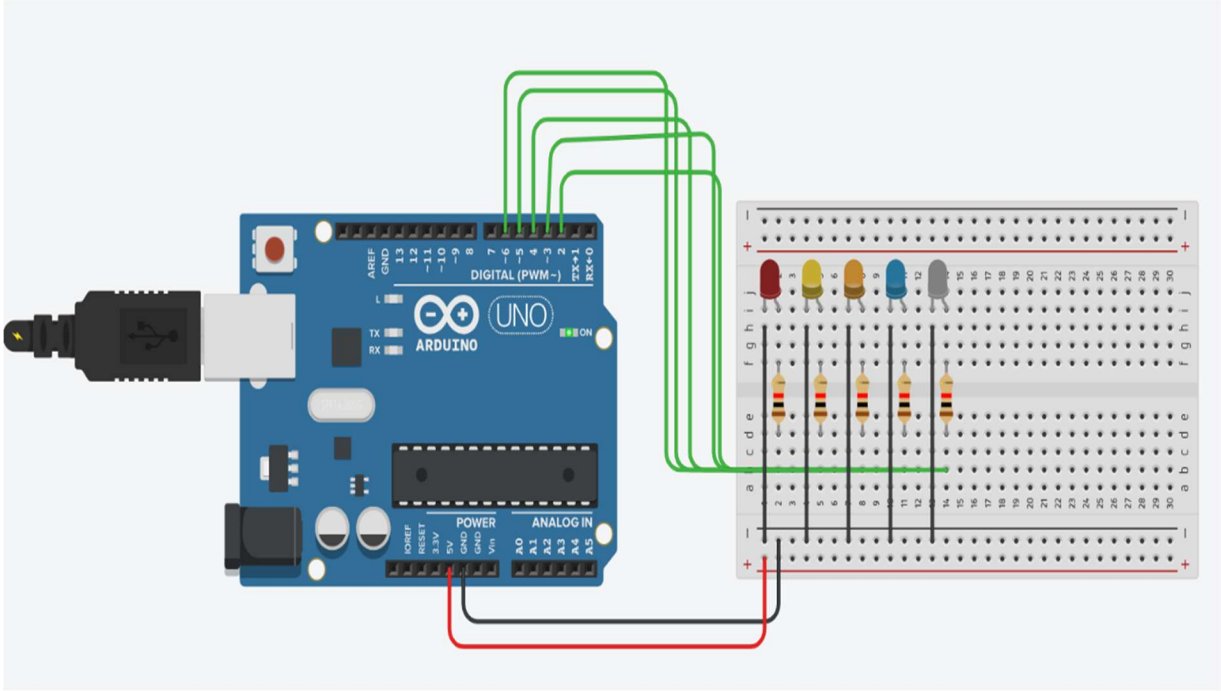
LED Control Systems:
8051 Microcontroller & Arduino UNO



1. Introduction & Objectives

This document presents two foundational embedded systems projects built to strengthen practical skills in microcontroller-based hardware control, moving from low-level assembly programming on the classic 8051 architecture to higher-level C programming on the Arduino platform. Both projects center on the same core concept — digital output control of LEDs — but approach it through different toolchains, languages, and levels of abstraction, offering a useful side-by-side view of embedded development at different layers of the stack.

The 8051 project was built using Keil uVision5 for code editing and assembly, with Proteus 8 Professional used for schematic capture and real-time circuit simulation. The Arduino project was designed and tested in Tinkercad Circuits, using the Arduino UNO board programmed in C/C++.

Presenting these projects together also makes a useful point for anyone reviewing the portfolio: the same underlying electrical problem — turning digital outputs on and off in a controlled sequence — looks very different depending on which layer of the stack you're working in. Seeing both the raw register-level version and the abstracted, library-driven version side by side makes that difference concrete rather than theoretical.

Objectives

- Understand GPIO (General Purpose Input/Output) control at the register level using 8051 assembly instructions (MOV, RR, ACALL, DJNZ, SJMP).
- Implement software-based timing delays using nested register loops.
- Design and simulate a working microcontroller circuit in Proteus without physical hardware.
- Translate the same conceptual goal (sequential LED control) into Arduino C using arrays and loops.
- Compare low-level (assembly) and high-level (C) approaches to the same hardware problem.
- Build a portfolio-ready, well-documented project pair suitable for GitHub and LinkedIn.

2. Project 1: 8051 LED Blink & Sequencing

2.1 Overview and Learning Goals

This project implements a rotating LED output pattern on Port 1 of an AT89C51 microcontroller, using pure 8051 assembly language. The core idea is to load an initial bit pattern into the accumulator, rotate it, and continuously write the rotated value back out to the port — creating a visual 'chasing' effect across the connected LEDs, with a software delay routine controlling the speed of rotation.

Learning goals for this project included:

- Getting comfortable with the 8051-instruction set and addressing modes.
- Understanding how rotate instructions (RR/RL) manipulate bit patterns for LED effects.
- Writing a calibrated, nested delay loop using general-purpose registers.
- Verifying logic through Keil's built-in disassembly and register-level debugging.
- Validating real-world behavior using Proteus circuit simulation.

2.2 Tools & Development Environment

Tool	Purpose
Keil uVision5	Assembly editing, compilation, hex generation, and step-through debugging
Proteus 8 Professional	Schematic capture and live circuit simulation
Microcontroller	AT89C51 (8051 family, 8-bit)
Output Devices	8x LEDs (Blue, Green, Orange, Pink, Purple, Red, White, Yellow) on Ports 1 & 2

The Keil project ('es') built cleanly with zero errors and zero warnings, producing a hex file ready for simulation:

```
Build target 'Target 1'
linking...
Program Size: data=8.0 xdata=0 code=31
creating hex file from ".\Objects\uart"...
".\Objects\uart" - 0 Error(s), 0 Warning(s).
Build Time Elapsed:  00:00:02
```

2.3 Circuit Design in Proteus

The schematic connects Port 1 (P1.0–P1.7) of the AT89C51 to eight LEDs of different colors, providing clear visual feedback on which output bits are active at any moment. A shared ground return line ties all LED cathodes together. This setup allows the rotate-and-output pattern in the assembly code to be observed directly as a moving 'lit' LED across the row.

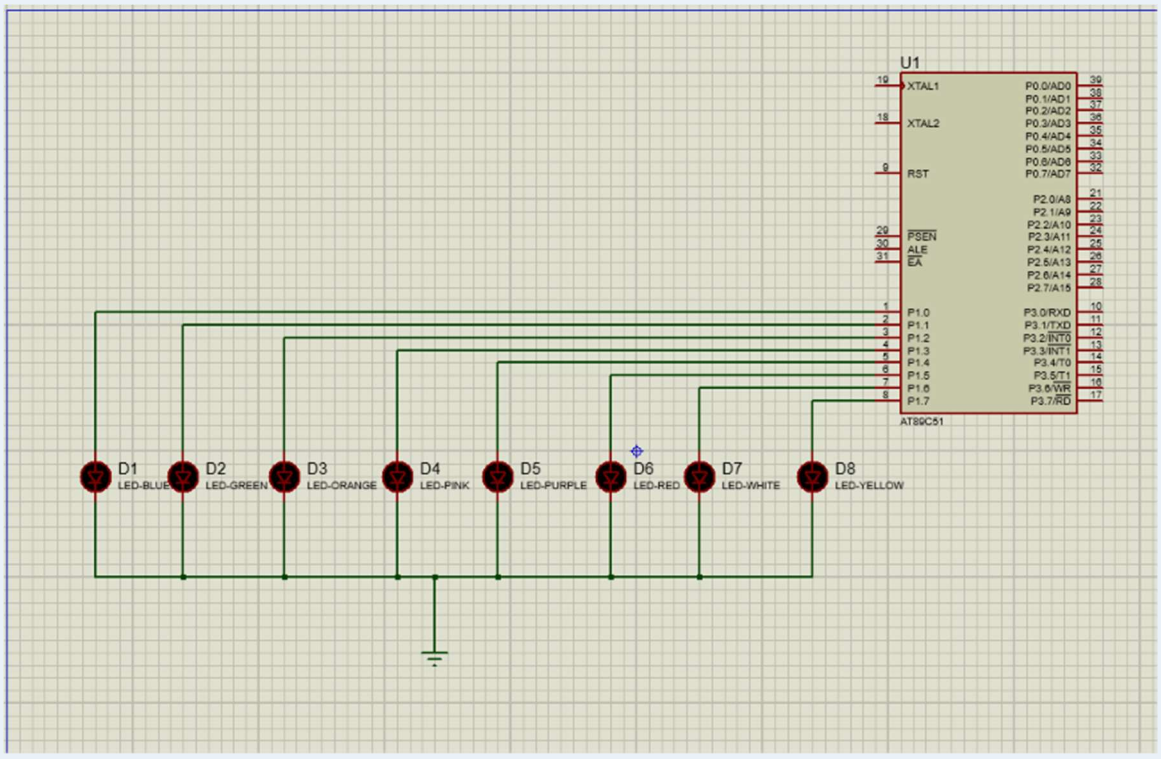


Fig. 1 — Proteus schematic: AT89C51 with 8 LEDs wired to Port 1

During simulation, the parallel port monitor confirms the live bit pattern being driven onto Port 1, matching the expected output of the rotate instruction at each step of the loop.

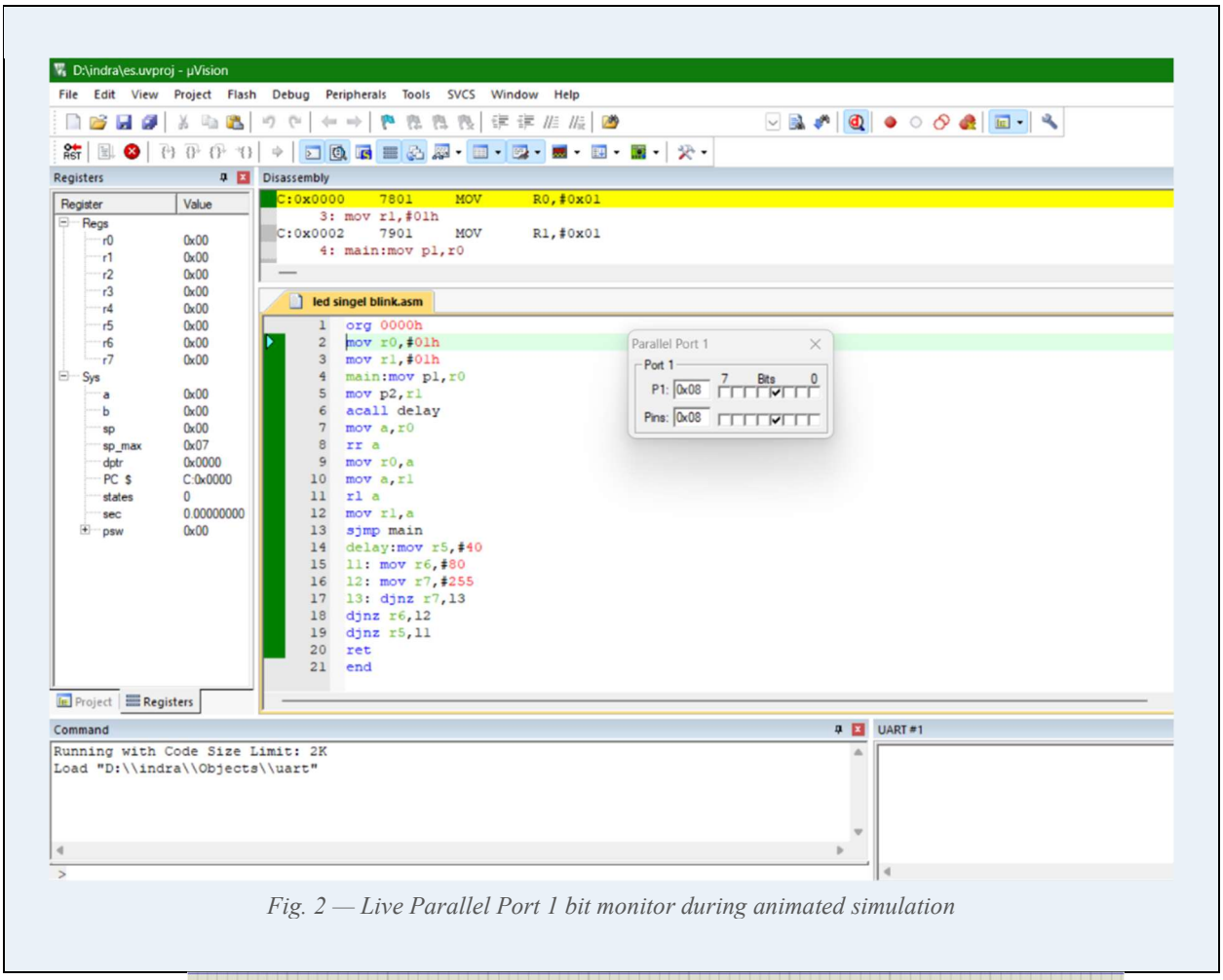


Fig. 2 — Live Parallel Port 1 bit monitor during animated simulation

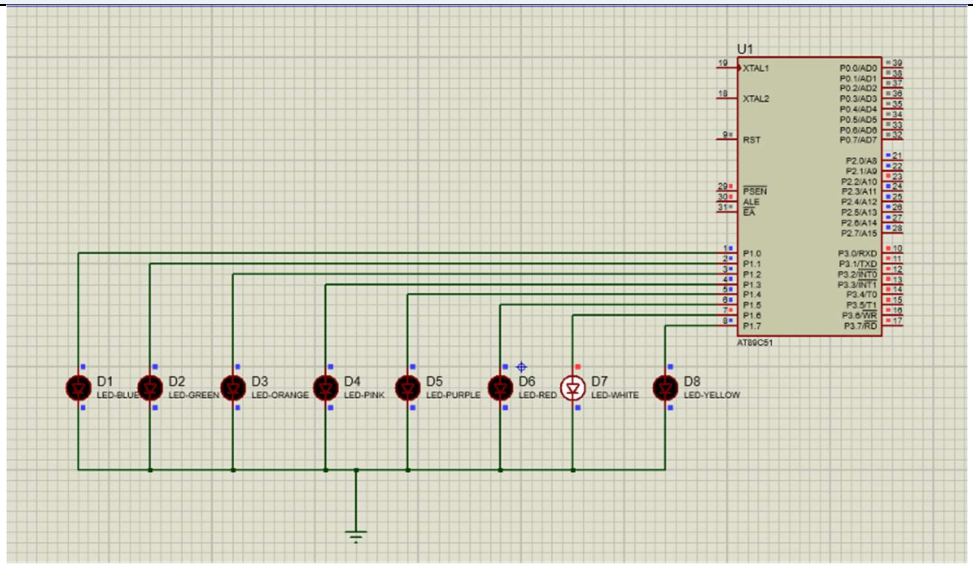


Fig. 2a — Simulation frame with a single LED lit (rotating pattern, step 1)

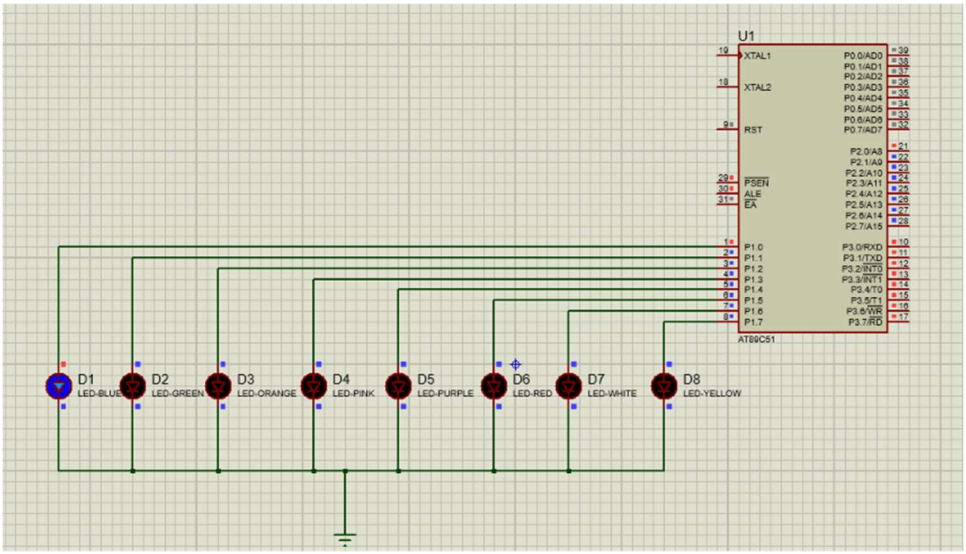


Fig. 2b — Simulation frame with a single LED lit (rotating pattern, step 2)

2.4 Assembly Code Walkthrough

Full source (led singel blink.asm):

```
org 0000h
mov r0,#01h
mov r1,#01h
main:mov p1,r0
mov p2,r1
acall delay
mov a,r0
rr a
mov r0,a
mov a,r1
rl a
mov r1,a
sjmp main
delay:mov r5,#40
l1: mov r6,#80
l2: mov r7,#255
l3: djnz r7,l3
djnz r6,l2
djnz r5,l1
ret
end
```

led singel blink.asm

```
1   org 0000h
2   mov r0,#01h
3   mov r1,#01h
4   main:mov p1,r0
5   mov p2,r1
6   acall delay
7   mov a,r0
8   rr a
9   mov r0,a
10  mov a,r1
11  rl a
12  mov r1,a
13  sjmp main
14  delay:mov r5,#40
15  l1: mov r6,#80
16  l2: mov r7,#255
17  l3: djnz r7,l3
18  djnz r6,l2
19  djnz r5,l1
20  ret
21  end
```

Fig. 1a — Keil uVision5 code editor view of led singel blink.asm

Line-by-line logic:

- R0 and R1 are initialized to 01h — the starting single-bit pattern for Ports 1 and 2 respectively.
- The main loop writes R0 to Port 1 and R1 to Port 2, lighting the corresponding LEDs.
- acall delay calls the timing subroutine so the pattern is visible before it shifts.
- rr a (rotate right) shifts the Port 1 bit pattern one position right each cycle, producing a right-moving LED effect.
- rl a (rotate left) shifts the Port 2 bit pattern left, producing the opposite direction on the second port.
- sjmp main creates an infinite loop, so the rotation runs continuously.
- The delay routine uses three nested DJNZ (decrement-and-jump-if-not-zero) loops (R5 × R6 × R7) to burn a fixed number of instruction cycles — a standard software-timing technique on the 8051, since it has no dedicated delay function.

2.5 Build & Simulation Results

The Keil disassembly view confirms the compiled machine code matches the source instructions exactly — for example, 'MOV R0,#0x01' assembles to opcode 7801, and 'MOV R1,#0x01' to 7901, verifying correct opcode generation before simulation.

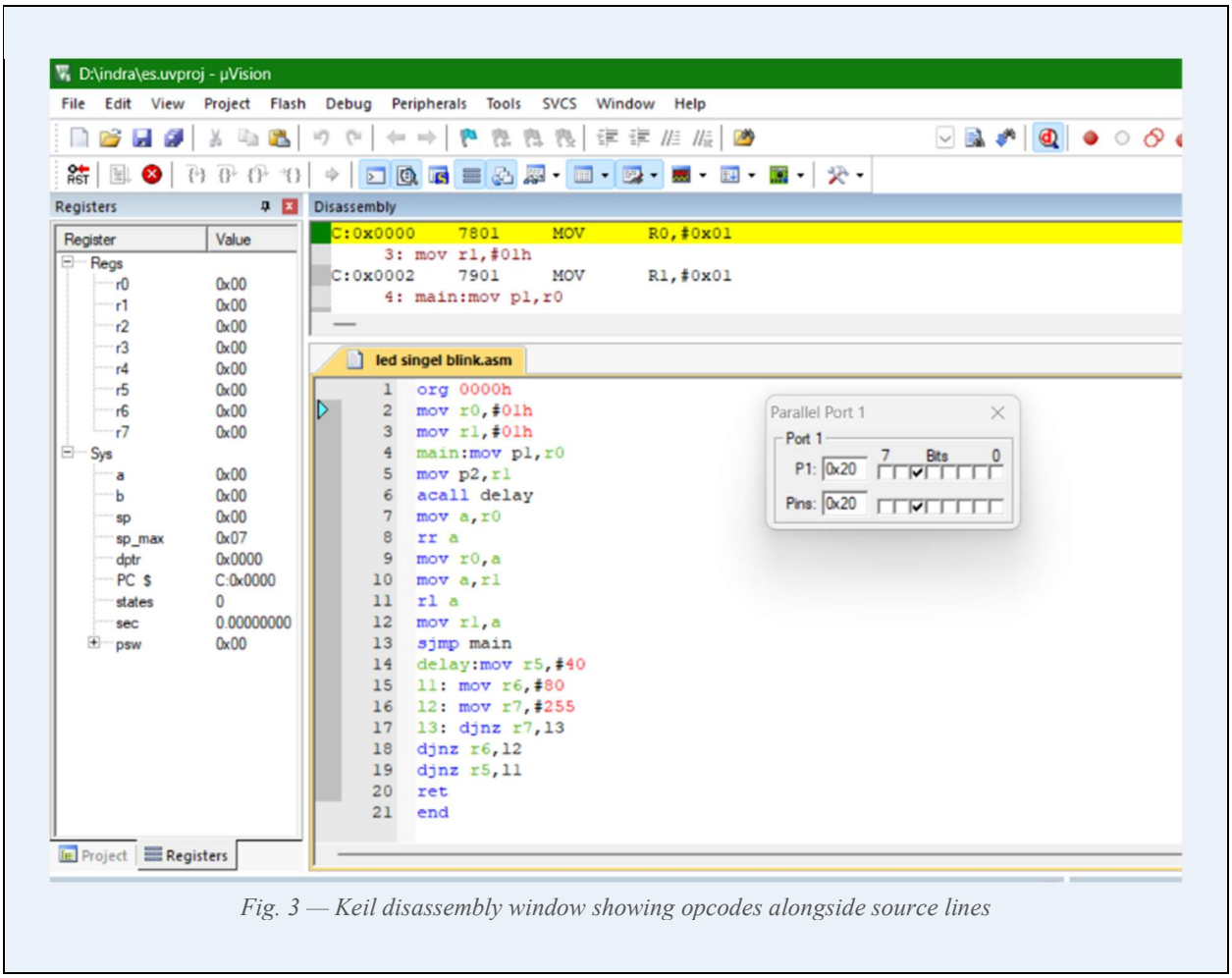


Fig. 3 — Keil disassembly window showing opcodes alongside source lines

In Proteus, the simulation ran successfully with the LED pattern visibly rotating across Port 1 in sync with the delay timing, confirming that the logic, timing, and hardware wiring all function together as intended. This end-to-end result — source code, verified opcodes, and a working simulated circuit — is the core deliverable of this project.

Debugging Notes

A few practical issues came up while getting this circuit to behave correctly, worth noting for future reference:

- Initial confusion between RR (rotate right, no carry involved) and RRC (rotate right through carry) — RR was correct here since no carry-flag dependency was needed for a pure visual rotation.
- Verifying the delay length required a couple of iterations — the first attempt used smaller register values and the rotation was too fast to see clearly in the Proteus animation.
- Confirming Port 2 (rl a path) rotated in the opposite direction from Port 1 (rr a path) was a deliberate design choice to visually distinguish the two ports during simulation, rather than a bug to fix.
- The parallel port bit monitor in Proteus (Fig. 2) was the fastest way to confirm the actual bit pattern being output, rather than relying only on the LED colors, which can be harder to read precisely frame-by-frame.

3. Project 2: Arduino LED Chase Effect

3.1 Overview and Learning Goals

This project implements a simple LED 'chase' effect using an Arduino UNO, where five LEDs light up one at a time in sequence, each staying on for a short interval before turning off and handing off to the next. Unlike the 8051 project's bit-rotation approach, this version uses an array of pin numbers and a straightforward for-loop in C++, reflecting the more abstracted, beginner-friendly style of Arduino development.

- Practice array-based management of multiple digital output pins.
- Reinforce the setup()/loop() structure fundamental to Arduino programming.
- Build and test a breadboard circuit virtually using Tinkercad before (optionally) wiring real hardware.
- Compare USB-powered vs external barrel-jack-powered circuit configurations.

3.2 Tools & Circuit Design

Tool	Purpose
Tinkercad Circuits	Virtual breadboard design and simulation
Board	Arduino UNO
Output Devices	5 LEDs (Red, Yellow, Orange, Blue, Grey) each with a current-limiting resistor
Power Options	USB (via computer) and external DC barrel jack, both tested

Each LED is wired through its own resistor to a dedicated digital pin (2–6) on the Arduino, with all cathodes returned to a shared ground rail on the breadboard. Two versions of the circuit were built to confirm the design works identically whether powered over USB or from an external adapter.

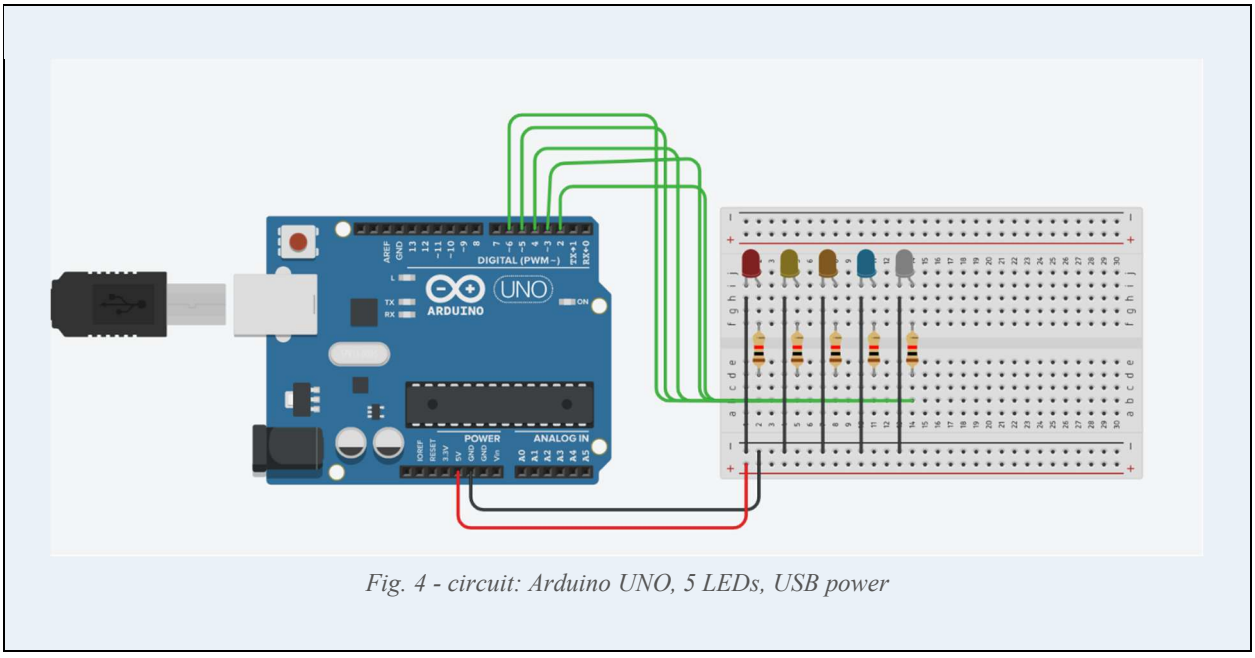


Fig. 4 - circuit: Arduino UNO, 5 LEDs, USB power

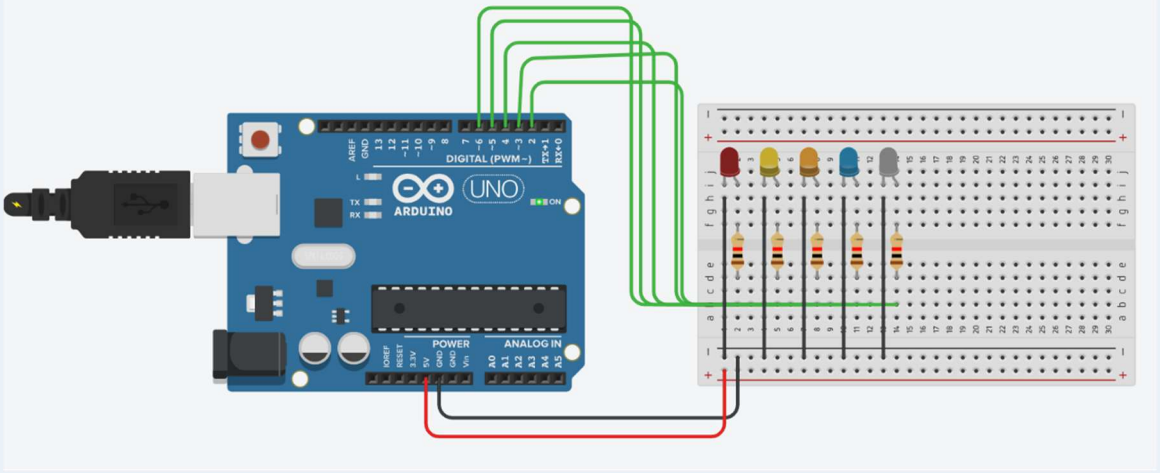


Fig. 5 — circuit powered externally via DC barrel jack

3.3 Code Walkthrough (C/C++)

```
const int ledPins[] = {2, 3, 4, 5, 6};
const int numLeds = 5;

void setup() {
  for (int i = 0; i < numLeds; i++) {
    pinMode(ledPins[i], OUTPUT);
  }
}

void loop() {
  // Chase effect: one LED on at a time
  for (int i = 0; i < numLeds; i++) {
    digitalWrite(ledPins[i], HIGH);
    delay(250);
    digitalWrite(ledPins[i], LOW);
  }
}
```

Logic breakdown:

- An array ledPins[] holds the five digital pin numbers, avoiding five separate variables.
- setup() loops through the array once at startup, configuring every pin as OUTPUT.
- loop() runs continuously: for each pin, it turns the LED on, waits 250 milliseconds, then turns it off before moving to the next index — producing the visual chase.
- Because the next LED only lights after the previous one is turned off, exactly one LED is illuminated at any given moment.

3.4 Behavior & Testing

The circuit was tested in confirming that the LEDs light in the correct order (pin 2 → 3 → 4 → 5 → 6) with even, consistent timing. The 250 ms delay was chosen to make the chase clearly visible without feeling sluggish — this value can easily be tuned by changing a single constant, one of the advantages of the array-driven approach over writing five near-identical blocks of code.

Design Choices & Alternatives Considered

A few implementation alternatives were considered before settling on the current array-based structure:

- Writing five separate digitalWrite() calls without a loop — rejected as repetitive and harder to maintain if the number of LEDs changes.
- Using millis()-based non-blocking timing instead of delay() — would allow other tasks to run concurrently, but was left out here since this project has no competing tasks; noted as a natural next step for a more advanced version.
- Adding a direction flag to reverse the chase — considered as a stretch goal, listed under Section 6 (Next Steps) rather than implemented in this version to keep the initial build focused and easy to verify.

4. Comparative Analysis: 8051 vs Arduino Workflow

Although both projects ultimately drive LEDs, the development experience differs substantially between the two platforms — one working close to the hardware in assembly, the other through a higher-level, library-assisted C environment.

Aspect	8051 (Assembly)	Arduino (C/C++)
Language level	Low-level, register-based	High-level, function-based
Timing control	Manual DJNZ delay loops	Built-in delay() function
Output control	Direct port/bit manipulation (rr, rl)	digitalWrite() per pin via array
Tooling	Keil uVision5 + Proteus simulation	Tinkercad virtual breadboard
Debugging	Disassembly + register/port inspection	Simulator visual + serial monitor (optional)
Learning curve	Steeper — requires instruction set knowledge	Gentler — abstracted hardware access
Best suited for	Understanding hardware fundamentals	Rapid prototyping and quick iteration

Working through both projects side by side reinforced how the same conceptual goal — controlled, sequential digital output — can be reached either by manually managing timing and bits at the register level, or by leaning on a runtime environment that abstracts those details away. Having built the register-level version first made the abstraction in the Arduino code much easier to appreciate and trust, rather than treating digitalWrite() and delay() as opaque magic.

5. Skills Demonstrated

Technical Skills

- 8051 assembly programming: data movement, arithmetic/rotate instructions, subroutine calls (ACALL/RET)
- Software delay-loop design using nested DJNZ counters
- GPIO/port-level output control on an 8-bit microcontroller
- Arduino C/C++ programming with arrays and iterative control structures
- Schematic capture and circuit simulation in Proteus 8 Professional
- Virtual breadboard prototyping in Tinkercad Circuits
- Reading and verifying compiled opcodes via disassembly in Keil uVision5
- Build-and-debug workflow: compiling, checking build logs, and validating output against source logic

Workflow & Documentation Skills

- Structuring a project for portfolio presentation (report, code, circuit diagrams together)
- Comparing architectures/tooling objectively to draw practical engineering conclusions
- Preparing hardware projects for public sharing on GitHub and LinkedIn

6. Conclusion & Next Steps

Both projects achieve the same visible outcome — a moving pattern of LED light — but by deliberately building the assembly version first, the underlying mechanics of timing, bit manipulation, and port control became concrete rather than abstract. Bringing the same idea to Arduino afterward highlighted exactly what a modern embedded framework automates on the programmer's behalf, and why that abstraction is valuable once the fundamentals are understood.

Planned extensions to this pair of projects:

- Add interrupt-driven control to the 8051 version (e.g., an external switch to reverse rotation direction).
- Port the Arduino chase effect to physical hardware and verify timing against the simulated version.
- Extend the Arduino sketch to accept pattern changes via serial input for interactive control.
- Combine both platforms in a future project — e.g., 8051 as a slave device communicating with an Arduino master over UART.

Together, these two builds form a compact but complete demonstration of embedded fundamentals: from raw register manipulation on a classic 8-bit microcontroller to modern, framework-assisted rapid prototyping — a useful pairing to showcase range across both ends of the embedded development spectrum.

Appendix A: 8051 Instruction Reference

The following instructions from the 8051-assembly project are commonly confused by beginners. This quick reference summarizes what each does, for anyone reviewing the code who is newer to 8051 assembly.

Instruction	Function
MOV	Copies a value between a register, memory location, or port
RR A	Rotates the accumulator's bits right by one position (wraps around)
RL A	Rotates the accumulator's bits left by one position (wraps around)
ACALL	Calls a subroutine within a 2KB address range, saving the return address
RET	Returns from a subroutine to the instruction after the calling ACALL/LCALL
SJMP	Short jump to a label within -128 to +127 bytes of the current instruction
DJNZ	Decrements a register and jumps to a label if the result is not zero
ORG	Assembler directive setting the starting address for the following code
END	Assembler directive marking the end of the source file

Understanding this small instruction set is enough to read and write a large share of simple 8051 programs — most beginner confusion comes from addressing modes and register naming rather than the instructions themselves.

Appendix B: Delay Timing Calculation

The 8051-delay subroutine uses three nested DJNZ loops with counters R5 (40), R6 (80), and R7 (255). Each DJNZ instruction takes a fixed number of machine cycles to execute, so the total delay can be approximated as:

```
Total loop iterations  = R5 x R6 x R7
                       = 40 x 80 x 255
                       = 816,000 iterations

At 12 MHz with 1 machine cycle = 1 microsecond (standard 8051 timing),
each DJNZ takes ~2 machine cycles, giving an approximate delay of:

  816,000 x 2 cycles x (1 / 12,000,000 s) ~= 0.136 seconds per loop pass
```

This is an approximation — actual timing also includes the overhead of the outer MOV instructions initializing R5–R7, and depends on the exact crystal frequency used in the target hardware or Proteus simulation profile. In practice, this delay value produced a rotation speed that was clearly visible and easy to follow during simulation, which was the primary goal rather than achieving a precise timing figure.

For the Arduino project, timing is far simpler to reason about: the delay(250) call blocks execution for exactly 250 milliseconds, since Arduino's core library abstracts away cycle-counting entirely — another clear illustration of the abstraction difference discussed in Section 4.